DELIVERABLE 1.2 / TECHNICAL BLUEPRINT

AML Temporal Robustness Lab

From pitch to prototype: financial graph learning under distribution shift

a. Problem Context and Project Summary

Financial crime investigators must prioritize suspicious activity across transaction networks. This project tests whether GraphSAGE is more or less robust than XGBoost and a non-graph MLP after the Elliptic step-43 shift, and whether recent-data adaptation improves recovery differently across model families. Its output will be a reproducible comparison and a Streamlit investigation-analysis interface. The task is retrospective classification of transactions associated with licit or illicit entities, not proof of money laundering or a production compliance decision. The revised contribution goes beyond repeating the original tree-versus-GCN result [1].

Revision in response to feedback

MLP is mandatory. The primary study crosses three model families with four adaptation policies. Expanding-history retraining controls for receiving additional labels, so rolling/recency gains cannot be attributed solely to having more labeled data. Temporal robustness and recovery are hypotheses to test, not promised findings. This is an original project experiment, not a claim that adaptation itself is new research.

b. Dataset

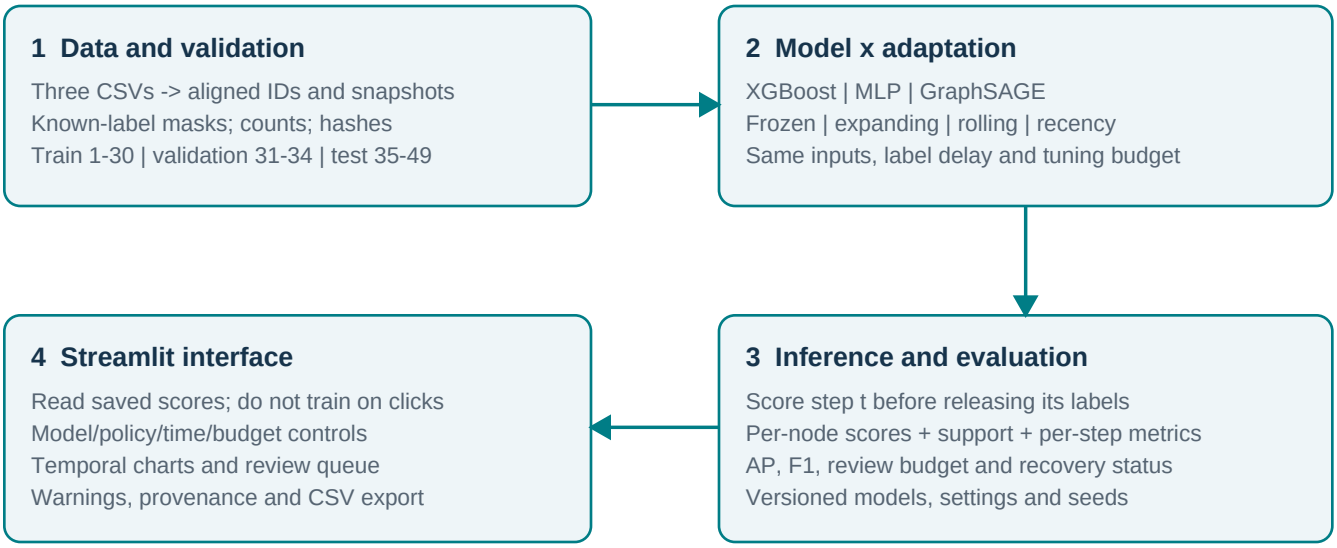
The original Elliptic release [2] is tabular data plus a directed Bitcoin transaction graph: 203,769 transaction nodes, 234,355 payment-flow edges and 49 snapshots. Published labels comprise 4,545 illicit, 42,019 licit and 157,205 unknown nodes. Illicit nodes are 2.2% of all nodes but 9.8% of known labels. Features include 94 local attributes (one is time step) and 72 supplied neighborhood aggregates. Feature names are mostly anonymized.

Access and formats. Download the publisher's release from Kaggle, extract the headerless feature CSV and the classes/edge CSVs into data/raw, and record provenance and checksums. The loader aligns IDs, verifies schema, labels, finite features, unique IDs/edges and within-step edge endpoints. It never silently substitutes sample data. Time step is metadata; the core predictors use the other 93 local features. Supplied aggregates enter a separate tree ablation.

Missing labels and privacy. Unknown nodes may supply snapshot context but are masked out of loss and metrics. Anonymization limits explanations and subgroup fairness analysis; do not attempt re-identification, redistribute credentials or describe predictions as legal findings. Original data is not bundled; publisher terms govern reuse.

Foundation status: fixture-based loader/EDA and protocol checks are executed. Original-data download was blocked in the authoring environment; real counts, a baseline drop, and full model/UI execution are not yet verified. The setup notebook and checklist specify the required local checks before submission.

c. Planned Architecture



Data flow: validated CSVs become aligned feature matrices, known-label masks and isolated graph snapshots. A common experiment runner will generate saved scores and metrics for all model-policy pairs. The interface reads those artifacts; interactive filters never retrain models or expose held-out labels to training.

Models and controls

XGBoost supplies a strong tabular baseline. The **MLP** uses two 64-unit hidden layers, ReLU and dropout. **GraphSAGE** uses two mean-aggregation layers with 64 hidden units and a linear binary head [3]. The MLP is the required non-graph neural control, with comparable depth, optimizer and training budget. Report parameter counts because architecture matching cannot perfectly remove capacity differences. Primary graph message passing follows incoming directed edges; any symmetrized graph is a secondary retrospective ablation.

Training and adaptation contract

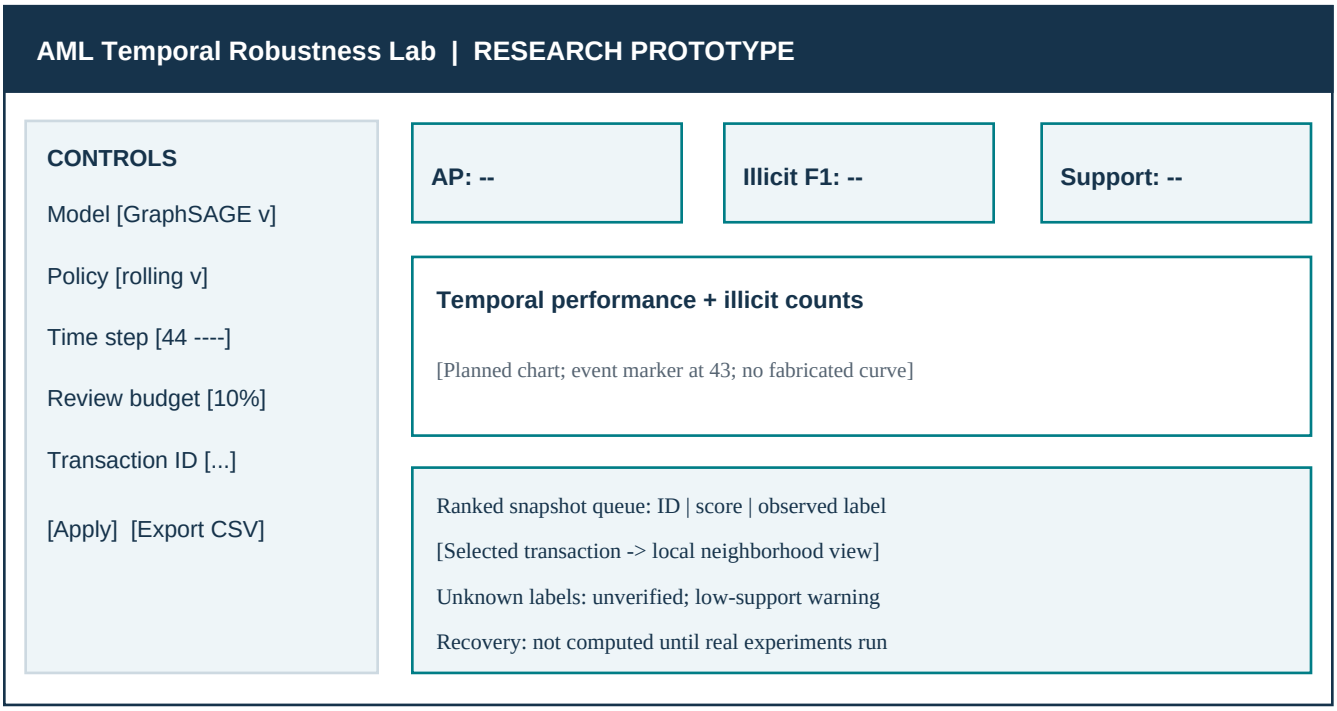
Initial training uses steps 1-30; validation uses 31-34; testing uses 35-49. Fit normalization and class weights on initial training data and keep them fixed. Select hyperparameters and a per-model F1 threshold on validation, then freeze them. During each holdout step t, refit only on eligible labels from steps less than t, score t, and release its existing known labels for t+1. This assumes a simulated one-step label delay, not measured operational turnaround. Unknown nodes never acquire invented labels.

Policy	Eligible history / purpose
Frozen	Keep the initial 1-30 model; drift reference.
Expanding	Refit on all labeled history before t; additional-data control.
Rolling	Refit on the preceding 8 steps only.
Recency	Refit on all past known labels; weight halves every 4 steps.

Use seeds 0/1/2, identical scoring nodes and fixed epoch/tuning budgets. No post-event tuning or current-test early stopping. Adaptive runs may use validation labels once historical; compare rolling/recency with expanding to isolate recency effects. Store model, policy, seed, cutoff, threshold and software versions beside every result. Python/pandas/scikit-learn support the pipeline; PyTorch/PyG, XGBoost and Streamlit comprise the planned full stack.

d. User Interface Plan

The planned Streamlit interface supports a reviewer comparing robustness experiments, rather than an unrestricted transaction-upload service. Inputs are a saved experiment/model version, adaptation policy, snapshot time step, review fraction and an optional transaction ID from that snapshot. Model files and result files come from the project pipeline, not untrusted user uploads.



Outputs and interactions

The completed interface will show per-step average precision (AP), illicit F1, labeled/illicit counts and a temporal chart with a marker at step 43. A comparison view will contrast policies and model families, with steps 35-42 and 44-49 summarized separately. A ranked queue displays scores, observed label status and transaction IDs. Selecting a node reveals its local neighborhood and graph degree; it does not imply a causal explanation of the score.

A review-budget slider changes only which saved scores are displayed and how many are reviewed. Evaluation precision@K/recall@K applies to known-label nodes only. Unknown nodes can appear in a separate, explicitly unverified queue, with no claimed precision. Low-support warnings and unavailable recovery estimates remain visible. CSV exports include experiment provenance and the research-use disclaimer.

Implementation state and acceptance checks

The supplied ui/app.py is an initial EDA prototype with an explicit sample/original-data selector, counts and a snapshot table. Model inference and recovery charts are planned; the UI does not fabricate predictions. The wireframe is a design artifact, not a screenshot of a completed model application.

Acceptance: a model/policy switch loads the matching artifact; a time-step change updates the queue and metrics; an absent ID produces a useful message; missing files produce a setup instruction; unknown labels remain unknown; reported metrics reproduce saved evaluator outputs. Cache only validated local data, and keep training out of the UI callback path.

e. Innovation and Anticipated Challenges

New experimental question and measurable outcomes

The original study already compared GCN and tree models, documented deterioration after step 43, and examined retraining [1]. This project therefore tests the **interaction between model family and adaptation policy**: do graph and non-graph models lose different amounts of performance, and does recent-history adaptation help one family more than another? MLP separates a neural-model comparison from the graph-specific question. No model superiority or recovery is assumed.

Primary outputs are mean per-step AP and changes between steps 35-42 and 44-49, with event step 43 reported separately. Report AP minus the per-step illicit prevalence as context for a random-ranking baseline, not a complete correction for changing difficulty. Compare rolling/recency minus expanding gains within each model, then compare those gains across families. Report seed means/ranges, support counts, fixed-threshold F1 and labeled precision/recall within a 10% review budget. AP means average precision, not trapezoidal PR-AUC.

Recovery speed (secondary): first post-event step attaining at least 90% of that model-policy's mean pre-event F1 for two consecutive calendar steps, each with at least 10 illicit and 10 licit labeled nodes. A zero reference or inadequate support makes this not estimable. If recovery is absent by 49, report "not observed by 49." Show the full trajectory because follow-up is short and the chosen criterion is descriptive. Do not infer causal shutdown effects from a performance drop.

Three risks and mitigation

Risk	Mitigation
Sparse/selected labels and noisy thresholds	Never label unknowns licit. Print validation illicit/licit counts for 31-34; use AP as primary, validate thresholds once, and display support by time step. Flag single-class windows and unreliable small groups; conclusions apply to labeled nodes.
Temporal leakage and unrealistic feedback	Score before using current labels; restrict graph context to the current available snapshot. Fix preprocessing and hyperparameters. One-step label availability is an explicit simulation. Supplied successor aggregates and snapshot access limit real-time claims.
Compute, scope and unstable recovery	One shallow GNN and three seeds, fixed epochs, bounded tuning and sampling if needed. Start on one snapshot. Retain MLP and adaptation comparisons; drop extensions first. Report missing/unevaluable steps rather than silently changing history.

Optional extensions after the core study

Feed training-only GraphSAGE embeddings into XGBoost, or analyze errors by degree/isolation. An illicit-neighbor breakdown is a retrospective diagnostic only: held-out labels must never become model inputs, and neighborhood label coverage must be reported. Elliptic++ and dynamic GNNs are outside the required scope.

f. Implementation Timeline to December 9

The requested six-week template exceeds the approximately four-week period remaining. The plan below uses six dated work blocks ending December 9; these are phases, not six full calendar weeks. Dates are proposed milestones, not claims of completed work.

Work block	Focus	Expected outcome
1 | Nov 11-15	Original-data access; environment; EDA; first frozen baseline	Executed real setup; validation support; illicit-count plot; measured pre/post-43 baseline trajectory.
2 | Nov 16-20	Core XGBoost, MLP and GraphSAGE; split enforcement	All three train and score end-to-end; shared features, masks and metric schema.
3 | Nov 21-25	Expanding, rolling and recency policies	Past-only refits; labels delayed one step; settings locked; one-seed pilot complete.
4 | Nov 26-30	Three-seed robustness and recovery analysis	Model-policy interaction tables; support-aware trajectories; recovery/censoring outputs.
5 | Dec 1-5	Streamlit artifact integration and refinement	Working controls, queue, temporal plots, warnings and export; optional diagnostic if time allows.
6 | Dec 6-9	Reproducibility, final report and demonstration	Fresh-run checks; final figures; documented limitations; stable demo and final submission.

Week-one gate: plot actual illicit counts per snapshot, print validation counts for 31-34, and check whether the measured frozen baseline deteriorates after 43. Count changes alone do not confirm a performance collapse. If access is still blocked, resolve it before claiming prototype readiness; fixture data cannot substitute for this check.

g. Responsible AI Reflection

Fairness and harm: selective labels and anonymized features prevent reliable demographic fairness claims. False alerts can harm legitimate users, so predictions support human investigation and must never trigger automatic sanctions. **Transparency:** distinguish unknown from licit, label sample output clearly, publish the temporal/label assumptions and retain unfavorable results. Attention or neighborhood association does not establish guilt. **Environmental cost:** use a shallow GNN, bounded searches, shared preprocessing and cached artifacts; record CPU/GPU, elapsed training time and available resource measurements rather than inventing an energy estimate.

References and assistance disclosure

[1] Weber et al. (2019), *Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics.* https://arxiv.org/abs/1908.02591
[2] Elliptic, publisher dataset: https://www.kaggle.com/datasets/ellipticco/elliptic-data-set
[3] PyTorch Geometric, SAGEConv documentation: https://pytorch-geometric.readthedocs.io/en/latest/generated/torch_geometric.nn.conv.SAGEConv.html

AI assistance supported design, writing, initial code generation and formatting. Original-data execution, full model-stack verification and empirical validation remain the student's responsibility. The repository checklist distinguishes executed fixture checks from pending work.